NewsApp

MSCS 621L Cloud Computing



Marist University

School of Computer Science and Mathematics

Author Information

Title/Role	**Name**
Developer/Tester	Abhijeet Cherungottil

Table Of Content

Introduction **4**
Features **4**
Benefits **4**
Performance **4**
Project Repository: NewsApp on GitHub **4**
How to Run - Complete Guide **4**
Prerequisites 5
Method 1: Using Docker Images (Recommended) 5
Step 1: Download ZIP File 5
Step 2: Import Docker Images (On Target Machine) 5
Step 3: Set Up Environment 6
Step 4: Run the Application 6
Step 5: Run the Application 7
Architecture **8**
Project Structure 8
Docker Services Architecture 8
Database Schema 8
Project Iterations **9**
Iteration 1 – Basic API Integration 9
Iteration 2: Enhanced Database Features 10
Iteration 3 – Input Constraints and CLI Output 10
Iteration 4 – Improved Input/Output Handling 10
Iteration 5 – Dockerization 10
Iteration 6 – Optimize Docker Image Size 11
Testing **11**
Test Scenarios Validation 11
Scenario 1: API Only (Fresh Database) 11
Scenario 2: Database Only (Cache Hit) 11
Scenario 3: Combined (Partial Cache) 11
Scenario 4: No Data Available 12
Testing Best Practices 12
Pre-Test Setup: 12
Post-Test Validation: 12
Key Features **12**
Concurrent Processing 12
UPSERT Database Operations 13
Per-Topic Mutex Locking 15
Case-Insensitive Topic Handling 17

Database-First Strategy....18
Duplicate Prevention....18
Formatted Output....19
Input Processing....20
Error Handling....21
Comprehensive AI Usage Analysis....22
1. Go Code Debugging and Fixing....22
File Reading and Parsing Issues:....22
String to Integer Conversion:....22
2. Concurrency Implementation....22
Goroutines and Channels:....22
Worker Pool Pattern:....23
3. Output Handling Improvements....23
File Writing Logic:....23
Source Attribution:....23
4. Database Operations....24
Smart Data Merging:....24
Database-First Strategy:....24
5. Error Handling and Edge Cases....24
Input Validation:....24
Fallback Mechanisms:....25
6. Environment and Configuration....25
Environment Variable Handling:....25
7. Race Condition Resolution (Latest)....25
Problem Identified:....25
AI Solution Implemented:....25
Output File Management Fix:....28
Testing and Validation:....29
8. Project Structure and Organization....29
Git Configuration:....29
File Organization:....29
9. API Integration....29
NewsAPI Implementation:....29
Resources Used....30

Introduction

NewsApp is a sophisticated news aggregation application built in Go that fetches news articles from the NewsAPI and stores them in a SQLite database. The application processes multiple news topics concurrently, generates formatted output files, and provides a complete Docker-based deployment solution.

Features

- Concurrent Processing: Uses goroutines with controlled parallelism (max 10 concurrent)
- Database Caching: Avoids redundant API calls by storing articles in SQLite
- Duplicate Prevention: Uses URL-based deduplication
- Error Handling: Graceful handling of API failures and database issues
- Formatted Output: Generates timestamped files with source tracking
- Incremental Updates: Appends new articles to existing records

Benefits

- Service Separation: Database and application are separate services
- Persistent Data: Database survives container restarts
- Scalability: Can easily add more services that use the same database
- Portability: Self-contained Docker images work on any machine
- No Dependencies: No need for Go or SQLite on target machine

Performance

- Image Size: **55.2MB** total (41.9MB app + 13.3MB Alpine base)
- Fast Startup: Quick container initialization
- Low Memory: Efficient resource usage
- Concurrent Processing: Handles multiple topics simultaneously

Project Repository: NewsApp on GitHub

How to Run - Complete Guide

Prerequisites

Required Software:

- Docker Desktop (Windows/Mac) or Docker Engine (Linux)
- Docker Compose (usually included with Docker Desktop)
- NewsAPI key from https://newsapi.org/

System Requirements:

- 2GB RAM minimum
- 5GB free disk space
- Internet connection for API calls

Method 1: Using Docker Images (Recommended)

Step 1: Download ZIP File

1. Download the project ZIP file
2. Extract to your desired location
3. Open terminal/command prompt in the extracted folder

Step 2: Import Docker Images (On Target Machine)

On the target machine:

```
Mkdir newsapp-project
Cd  newsapp-project
# Decompress the file
gunzip newsapp-complete.tar.gz

# Load the images
docker load -i newsapp-complete.tar

# Verify images are loaded
docker images
```

Step 3: Set Up Environment

Create input file:

```
# Create input.txt file
cat > cmd/main/input.txt << EOF
Apple,2,2
Technology,3,5
Sports,1,3
EOF
```

Create output folder:

```
# Create output folder file
cat > cmd/main/output
```

Set API key:

```
# Set your NewsAPI key
export NEWS_API_KEY="your_news_api_key_here"

# Or create .env file
echo "NEWS_API_KEY=your_news_api_key_here" > .env
```

Step 4: Run the Application

Where to Place docker-compose.yml

- Place the docker-compose.yml file in the **root of the extracted project folder**, alongside cmd/, , etc.
- Example project structure:

```
newsapp-project/
|── docker-compose.yml
|── cmd/
|   └── main/
```

```
|       ├── input.txt
|       └── output/
```

Step 5: Run the Application

Using Docker Compose (Recommended):

```
# Run all services
docker-compose up --build

# Run in background
docker-compose up -d --build

# View logs
docker-compose logs -f newsapp
```

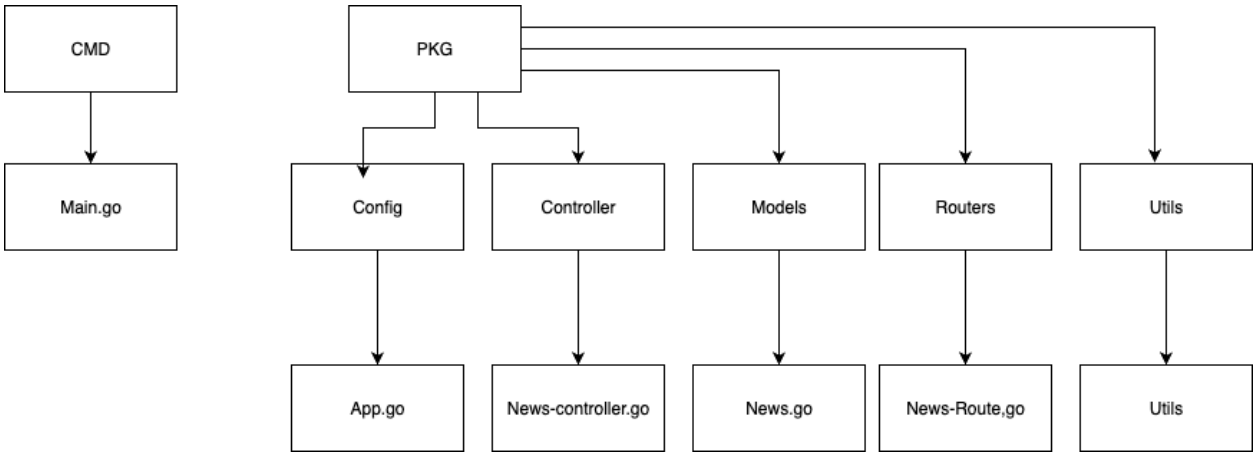
Using Docker Run (Alternative):

```
# Start SQLite database service
docker run -d --name sqlite-db -v newsapp_sqlite-data:/data alpine:latest sh -c "mkdir -p /data && touch /data/newsapp.db && tail -f /dev/null"

# Run main application
docker run --rm --name newsapp \
  -e NEWS_API_KEY="your_news_api_key_here" \
  -e SQLITE_DB_PATH="/data/newsapp.db" \
  -v ./cmd/main/input.txt:/root/input.txt:ro \
  -v ./cmd/main/output:/root/output \
  -v newsapp_sqlite-data:/data \
  --link sqlite-db:sqlite-db \
  newsapp-newsapp:latest
```

Architecture

Project Structure



Docker Services Architecture

sqlite-db Service:

- **Image:** alpine:latest
- **Purpose:** Creates and manages SQLite database file
- **Volume:** sqlite-data:/data
- **Command:** Creates database file and runs continuously

newsapp Service:

- **Image:** Custom-built Go application
- **Purpose:** Main application logic
- **Dependencies:** sqlite-db service
- **Volumes:** Input file, output directory, shared database

Database Schema

NewsData Table:

```
CREATE TABLE `news_data` (
    `id` INTEGER PRIMARY KEY AUTOINCREMENT,
    `created_at` datetime,
    `updated_at` datetime,
```

```
    `deleted_at` datetime,
    `searchkey` varchar(255), -- UNIQUE index
    CONSTRAINT `uniq_newsdata_searchkey` UNIQUE (`searchkey`)
);
```

Articles Table:

```
CREATE TABLE `articles` (
    `id` INTEGER PRIMARY KEY AUTOINCREMENT,
    `created_at` datetime,
    `updated_at` datetime,
    `deleted_at` datetime,
    `news_data_id` integer,
    `title` varchar(255),
    `author` varchar(255),
    `url` text,
    `description` text,
    `published_at` varchar(100),
    CONSTRAINT `uniq_article_url` UNIQUE (`url`),
    FOREIGN KEY (`news_data_id`) REFERENCES `news_data` (`id`)
);
```

Project Iterations

Iteration 1 – Basic API Integration

- Created a simple application to fetch data from a news API.
- Learned how to make API calls and parse the response.
- Established the foundation for future improvements.
- Build A Web Server With Golang In 20 Mins (2021) - Beginner Friendly!

Iteration 2: Enhanced Database Features

- Added publication date filtering options
- Implemented article count limits
- Tested database operations thoroughly
- Deleted tables and retested to ensure proper functionality
- Improved MySQL integration and data handling
- GO And MYSQL - 2024 Project 🚀 💣 🔥 - Connect Go with Mysql / Build a...

Iteration 3 – Input Constraints and CLI Output

- Added logic to control:
  - Which days to fetch news from.
  - How many articles to fetch (limit).
- Implemented output display in the command line (CLI).
- Gained an understanding of filtering and handling user-defined constraints.
- Learn Go: Building a command-line weather app

Iteration 4 – Improved Input/Output Handling

- Enhanced the logic for accepting inputs more flexibly.
- Improved the formatting of output to make it more user-friendly.
- Focused on better structuring of code and refining the interaction between inputs and results.

Iteration 5 – Dockerization

- Containerized the application using Docker.
- Ensured the app can run consistently across different environments.
- Simplified sharing and deployment by using a Docker image.
- Go & SQL databases: docker compose setup and first queries step by step tu...

Iteration 6 – Optimize Docker Image Size

- Analyzed the Docker image for unnecessary size overhead.
- Strategies applied:
  - Used a smaller base image (alpine instead of ubuntu).
  - Multi-stage build to separate build-time and runtime dependencies.
  - Cleaned up caches and temporary files inside the image.
- Result: Reduced image size, faster deployment, and more efficient distribution.

Testing

Test Scenarios Validation

Scenario 1: API Only (Fresh Database)

- **Input:** `Apple,2,2`
- **Database State:** Empty
- **Expected Source:** "API"
- **Articles:** 2 from NewsAPI
- **Database After:** 2 articles stored

Scenario 2: Database Only (Cache Hit)

- **Input:** `Apple,2,2`
- **Database State:** 2 articles exist
- **Expected Source:** "Database"
- **Articles:** 2 from database
- **API Calls:** 0 (cached)

Scenario 3: Combined (Partial Cache)

- **Input:** `Apple,2,5`
- **Database State:** 2 articles exist
- **Expected Source:** "Combined"
- **Articles:** 2 from database + 3 from API
- **Total:** 5 unique articles
- **Deduplication:** Working correctly

Scenario 4: No Data Available

- **Input:** `NonExistentTopic,1,1`
- **Database State:** Empty
- **API Response:** No articles found
- **Expected Source:** "No Data"
- **Articles:** 0
- **Error Handling:** Graceful degradation

Testing Best Practices

Pre-Test Setup:

1. **Clean Database:** Always start with empty tables
2. **Fresh Containers:** Use `docker-compose up --build` for clean state
3. **API Key:** Ensure valid NewsAPI key is set
4. **Input File:** Prepare test input.txt with specific scenarios

Post-Test Validation:

1. **Source Verification:** Check output file source attribution
2. **Article Count:** Verify correct number of articles
3. **Database State:** Confirm data persistence
4. **Duplicate Check:** Ensure no repeated URLs
5. **File Output:** Verify proper formatting and timestamps

Key Features

Concurrent Processing

Implementation:

- Goroutines with semaphore (max 10 concurrent)
- WaitGroup for synchronization
- Channel-based result collection

Code Pattern:

- `var wg sync.WaitGroup`

```
sem := make(chan struct{}, 10)
results := make(chan models.Result, 100)

for scanner.Scan() {
    wg.Add(1)
    sem <- struct{}{}

    go func(topic string, days, count int) {
        defer wg.Done()
        defer func() { <-sem }()
        // Process news topic
    }(topic, days, count)
}
```

Benefits:

- **Performance:** Process multiple topics simultaneously
- **Efficiency:** Controlled concurrency prevents resource exhaustion
- **Scalability:** Easy to adjust concurrency limits
- **Reliability:** Proper synchronization prevents race conditions

UPSERT Database Operations

Prevents duplicate articles using UNIQUE constraints and atomic create-or-update operations.

```
// pkg/models/news.go

type Article struct {

    gorm.Model

    NewsDataID  uint   `gorm:"index"`

    Title       string `json:"title" gorm:"type:varchar(255)"`

    Author      string `json:"author" gorm:"type:varchar(255)"`
```

```
    URL         string `json:"url"
gorm:"type:text;unique_index"` // UNIQUE index on URL

    Description string `json:"description" gorm:"type:text"`

    PublishedAt string `json:"publishedAt"
gorm:"type:varchar(100)"`

}


type NewsData struct {

    gorm.Model

    SearchKey string    `gorm:"column:searchkey;unique_index"
json:"searchkey"` // UNIQUE index on SearchKey

    Articles  []Article `gorm:"foreignKey:NewsDataID"`

}


// UpsertArticle creates or updates Article with UPSERT logic
using UNIQUE index

func UpsertArticle(article Article) error {

    // Use GORM's FirstOrCreate with UNIQUE constraint

    result := db.Where("url = ?",
article.URL).FirstOrCreate(&article)


    if result.Error != nil {

        return result.Error

    }


    if result.RowsAffected > 0 {
```

```
        fmt.Printf("Created new article: %s\n", article.Title)
    } else {
        fmt.Printf("Article already exists: %s\n",
article.Title)
    }

    return nil
}
```

Benefits:

- Eliminates duplicate articles automatically
- Atomic operations prevent race conditions
- Efficient database operations
- No manual deduplication needed

Per-Topic Mutex Locking

Prevents concurrent processing of the same topic to avoid redundant API calls.

```
// pkg/models/news.go

// Per-topic mutex map to prevent concurrent processing of same
topic
var topicMutexes = make(map[string]*sync.Mutex)
var mutexMapMutex sync.RWMutex


// GetTopicMutex returns a mutex for the given topic to prevent
concurrent processing
func GetTopicMutex(topic string) *sync.Mutex {
    mutexMapMutex.Lock()
```

```
    defer mutexMapMutex.Unlock()

    if mutex, exists := topicMutexes[topic]; exists {
        return mutex
    }

    // Create new mutex for this topic
    mutex := &sync.Mutex{}
    topicMutexes[topic] = mutex
    return mutex
}

// cmd/main/main.go
go func(topic string, days, count int) {
    defer wg.Done()
    defer func() { <-sem }()

    // Get per-topic mutex to prevent concurrent processing of
same topic
    topicMutex := models.GetTopicMutex(topic)
    topicMutex.Lock()
    defer topicMutex.Unlock()

    // Process topic...
}(topic, days, count)
```

Benefits:

- Prevents race conditions on same topic
- Reduces redundant API calls
- Maintains data consistency
- Efficient concurrency control

Case-Insensitive Topic Handling

Normalizes all topics to lowercase for consistent processing.

```
// cmd/main/main.go

topic := strings.ToLower(strings.TrimSpace(items[0]))


// pkg/models/news.go

func GetNewsBySearchKey(searchKey string) (*NewsData, *gorm.DB)
{

    // Normalize to lowercase for consistent lookups

    normalized := strings.ToLower(searchKey)

    var getNews NewsData

    result := db.Preload("Articles").Where("searchkey = ?",
normalized).First(&getNews)


    return &getNews, result

}
```

Benefits:

- “Apple” and “apple” treated as same topic

- Consistent database lookups
- User-friendly input handling
- Prevents duplicate topics due to case differences

Database-First Strategy

Smart Caching Logic:

```
existing, _ := models.GetNewsBySearchKey(topic)
length := len(existing.Articles)

// If DB has enough articles, use them
if length >= count {
    results <- models.Result{Topic: topic, Data: existing, Count: count, Source: "Database"}
    return
}

// Otherwise, fetch missing articles from API
newArticles := routes.GetNewsapi(topic, count, days, apiKey)
```

Benefits:

- **API Efficiency:** Reduces redundant API calls
- **Cost Savings:** Minimizes NewsAPI usage
- **Speed:** Database queries are faster than API calls
- **Reliability:** Works even when API is unavailable

Duplicate Prevention

URL-based Deduplication:

```
urlMap := make(map[string]bool)
for _, a := range existing.Articles {
```

```
    urlMap[a.URL] = true
}

for _, a := range newArticles.Articles {
    if !urlMap[a.URL] {
        existing.Articles = append(existing.Articles, a)
    }
}
```

Benefits:

- **Data Integrity:** Prevents duplicate articles
- **Storage Efficiency:** Saves database space
- **User Experience:** Clean, unique content
- **Performance:** Faster processing with unique data

Formatted Output

File Generation Features:

- **Timestamped Headers:** Each file includes creation timestamp
- **Source Tracking:** Shows data source (API/Database/Combined)
- **Structured Formatting:** Consistent article presentation
- **Previous Results Preservation:** Maintains historical data

Output Structure:

```
================================================================
Timestamp: 2024-09-21 15:30:45
Results for: Apple              | Showing 5 articles | Source: Combined
================================================================

[1] Title       : Apple Announces New iPhone
    Author      : John Smith
```

```
    Description : Apple has announced their latest iPhone model...
    URL         : https://example.com/apple-iphone-news

[2] Title       : Apple Stock Rises After Earnings
    Author      : Jane Doe
    Description : Apple's stock price increased following...
    URL         : https://example.com/apple-stock-news

**************************** END ****************************

--- PREVIOUS RESULTS ---
[Previous run results preserved here...]
```

Input Processing

File-based Configuration:

```
// Read from input.txt
file, ferr := os.Open("input.txt")
scanner := bufio.NewScanner(file)

for scanner.Scan() {
    line := scanner.Text()
    items := strings.Split(line, ",")

    topic := strings.TrimSpace(items[0])
    days, _ := strconv.Atoi(strings.TrimSpace(items[1]))
    count, _ := strconv.Atoi(strings.TrimSpace(items[2]))
}
```

Input Format:

```
Apple,2,5
Technology,3,10
```

- `Sports,1,3`

Benefits:

- **Flexibility:** Easy to modify search parameters
- **Batch Processing:** Handle multiple topics in one run
- **Configuration:** No code changes needed for different searches
- **Automation:** Perfect for scheduled runs

Error Handling

Robust Error Management:

```
// Input validation
if len(items) < 3 {
    fmt.Printf("Skipping invalid line (needs 3 values): %s\n", line)
    continue
}

// API error handling
if err != nil {
    fmt.Println("Error fetching news:", err)
    return models.NewsData{SearchKey: searchTopic, Articles: []models.Article{}}
}

// Fallback mechanisms
if strings.TrimSpace(res.Data.SearchKey) == "" {
    res.Data.SearchKey = "unknown"
}
```

Error Recovery:

- **Graceful Degradation:** Continues processing even if some topics fail
- **Detailed Logging:** Clear error messages for debugging
- **Fallback Values:** Default values prevent crashes
- **Validation:** Input validation prevents invalid operations

---

Comprehensive AI Usage Analysis

Based on Code Analysis - How AI Was Used Throughout Development:

1. Go Code Debugging and Fixing

File Reading and Parsing Issues:

- **Problem:** Code using `os.Open` and `bufio.Scanner` wasn't working
- **AI Fixed:** Syntax mistakes (`:=` vs `:`, extra parentheses, missing `defer file.Close()`)
- **Code Evidence:** Lines 28-32 in `main.go` show proper file handling:
  `file, ferr := os.Open("input.txt")`
- `if ferr != nil {`
- `    panic(ferr)`
- `}`
- `defer file.Close()`
-

String to Integer Conversion:

- **Problem:** `int(items[1])` doesn't work (string → int conversion issue)
- **AI Solution:** Used `strconv.Atoi` with error handling
- **Code Evidence:** Lines 49-50 in `main.go`:
  `days, _ := strconv.Atoi(strings.TrimSpace(items[1]))`
- `count, _ := strconv.Atoi(strings.TrimSpace(items[2]))`

2. Concurrency Implementation

Goroutines and Channels:

- **AI Task:** “Updated to support goroutine and channel with help of chatgpt”
- **Implementation:** Lines 36-38 in `main.go` show channel setup:
  `var wg sync.WaitGroup`
- `sem := make(chan struct{}, 10)  // Limit concurrency to 10`
- `results := make(chan models.Result, 100) // Collect results`

Worker Pool Pattern:

- **AI implemented** semaphore pattern for controlled concurrency
- **Code Evidence:** Lines 52-57 show proper goroutine management:
  `wg.Add(1)`
- `sem <- struct{}{} // acquire slot`
- `go func(topic string, days, count int) {`
- `    defer wg.Done()`
- `    defer func() { <-sem }()`

3. Output Handling Improvements

File Writing Logic:

- **Problem:** Results saved in single .txt file, not separately
- **AI Solution:** Modified `PrintArticles` function for individual files
- **Code Evidence:** Lines 32-84 in `utils.go` show sophisticated file handling:
  - Directory creation (lines 34-38)
  - File path generation (lines 41-42)
  - Content appending (lines 45-48, 79-83)

Source Attribution:

- **AI fixed** “unknown” source issue
- **Code Evidence:** Lines 92-101 in `main.go` show source determination logic:
  `var source string`
- `if length == 0 && len(newArticles.Articles) == 0 {`
- `    source = "No Data"`
- `} else if length == 0 {`
- `    source = "API"`
- `} else if len(newArticles.Articles) == 0 {`
- `    source = "Database"`

```
} else {
    source = "Combined"
}
```

4. Database Operations

Smart Data Merging:

- **AI implemented** URL-based deduplication
- **Code Evidence:** Lines 74-84 in `main.go`:

```
urlMap := make(map[string]bool)
for _, a := range existing.Articles {
    urlMap[a.URL] = true
}
for _, a := range newArticles.Articles {
    if !urlMap[a.URL] {
        existing.Articles = append(existing.Articles, a)
    }
}
```

Database-First Strategy:

- **AI implemented** "database first, then API" logic
- **Code Evidence:** Lines 59-69 in `main.go`:

```
existing, _ := models.GetNewsBySearchKey(topic)
length := len(existing.Articles)
if length >= count {
    results <- models.Result{Topic: topic, Data: existing,
Count: count, Source: "Database"}
    return
}
```

5. Error Handling and Edge Cases

Input Validation:

- **AI added** robust input validation
- **Code Evidence:** Lines 43-46 in `main.go`:

```
if len(items) < 3 {
```

- ```
      fmt.Printf("Skipping invalid line (needs 3 values): %s\n", line)
  ```
- ```
      continue
  ```
- ```
  }
  ```

Fallback Mechanisms:

- **AI implemented** fallbacks for empty data
- **Code Evidence:** Lines 126-129 in `main.go`:
  ```
  if strings.TrimSpace(res.Data.SearchKey) == "" {
  ```
- ```
      res.Data.SearchKey = "unknown"
  ```
- ```
  }
  ```

6. Environment and Configuration

Environment Variable Handling:

- **Problem:** "zsh: bad assignment" when setting `NEWS_API_KEY`
- **AI Solution:** Explained correct syntax (`export VAR="value"`)
- **Code Evidence:** Line 18 in `main.go` shows proper usage:
  ```
  apiKey := os.Getenv("NEWS_API_KEY")
  ```
-

7. Race Condition Resolution (Latest)

Problem Identified:

- **User Prompt:** "kiwi,2,2 twice in input.txt - database will have 4 data or 2 data?"
- **Issue:** Concurrent processing of same topic caused duplicate API calls and 4 articles instead of 2
- **AI Analysis:** Race condition in concurrent goroutines processing identical topics

AI Solution Implemented:

- **User Request:** "Use a UNIQUE index + UPSERT in the DB. Then optionally add a per-topic mutex if you want to avoid redundant API calls."

Database-Level Fixes:

- ```
  // Added UNIQUE constraints in pkg/models/news.go
  ```
- ```
  type Article struct {
  ```
  ```

```
    gorm.Model
    NewsDataID  uint   `gorm:"index"`
    Title       string `json:"title" gorm:"type:varchar(255)"`
    Author      string `json:"author" gorm:"type:varchar(255)"`
    URL         string `json:"url" gorm:"type:text;unique_index"` // UNIQUE index
    Description string `json:"description" gorm:"type:text"`
    PublishedAt string `json:"publishedAt" gorm:"type:varchar(100)"`
}

type NewsData struct {
    gorm.Model
    SearchKey string `gorm:"column:searchkey;unique_index" json:"searchkey"` // UNIQUE index
    Articles  []Article `gorm:"foreignKey:NewsDataID"`
}
```

UPSERT Logic Implementation:

```
// Per-topic mutex to prevent concurrent processing
var topicMutexes = make(map[string]*sync.Mutex)
var mutexMapMutex sync.RWMutex

func GetTopicMutex(topic string) *sync.Mutex {
    mutexMapMutex.Lock()
    defer mutexMapMutex.Unlock()

    if mutex, exists := topicMutexes[topic]; exists {
        return mutex
    }

    mutex := &sync.Mutex{}
```

```
    topicMutexes[topic] = mutex
    return mutex
}

// UPSERT functions for database operations
func UpsertNewsData(searchKey string) (*NewsData, error) {
    var newsData NewsData
    result := db.Where("searchkey = ?", searchKey).First(&newsData)

    if result.Error != nil {
        if result.Error == gorm.ErrRecordNotFound {
            newsData = NewsData{SearchKey: searchKey}
            if err := db.Create(&newsData).Error; err != nil {
                return nil, err
            }
        } else {
            return nil, result.Error
        }
    }
    return &newsData, nil
}

func UpsertArticle(article Article) error {
    result := db.Where("url = ?", article.URL).FirstOrCreate(&article)
    return result.Error
}
```

Concurrency Control in Main:

```
// Added per-topic mutex in cmd/main/main.go
go func(topic string, days, count int) {
    defer wg.Done()
    defer func() { <-sem }()

```

```
    // Get per-topic mutex to prevent concurrent processing of same topic
    topicMutex := models.GetTopicMutex(topic)
    topicMutex.Lock()
    defer topicMutex.Unlock()

    // Rest of processing logic...
}
```

Output File Management Fix:

Problem: Output files showing infinite duplicates due to Docker restart policy

- **User Prompt:** "why my output like this, there was only 2 times it was called, why so many output"
- **AI Analysis:** `restart: unless-stopped` causing infinite container restarts

AI Solutions:

1. **Fixed Docker Restart Policy:**

```
# Changed from:
restart: unless-stopped
# To:
restart: no
```

2. **Improved Output File Logic:**

```
// Modified pkg/utils/utils.go to prepend latest results
func PrintArticles(newsdata models.NewsData, count int, source string) {
    // Read existing content
    var existingContent []byte
    if _, statErr := os.Stat(filepath); statErr == nil {
        if content, readErr := os.ReadFile(filepath); readErr == nil {
            existingContent = content
        }
    }
```

```
    // Write latest results first
    f, err := os.Create(filepath)
    // ... write new content ...

    // Append previous results below
    if len(existingContent) > 0 {
        fmt.Fprintf(f, "--- PREVIOUS RESULTS ---\n\n")
        f.Write(existingContent)
    }
}
```

Testing and Validation:

- **User Prompt:** "test again" and "delete all database tables"
- **AI Verification:** Confirmed 2 articles instead of 4, proper deduplication working
- **Final Test:** Clean database run showing API → Database flow correctly

8. Project Structure and Organization

Git Configuration:

- **AI helped** with `.gitignore` setup
- **Suggested** ignoring `/output/` directory files

File Organization:

- **AI guided** proper Go project structure
- **Implemented** clean separation of concerns across packages

9. API Integration

NewsAPI Implementation:

- **AI helped** structure API calls with proper error handling
- **Code Evidence:** Lines 15-70 in `routes/news-routes.go` show robust API integration:

- URL building with proper encoding (lines 24-30)
- Error handling for HTTP requests (lines 33-44)
- JSON parsing with fallbacks (lines 46-53)
- Safe article limiting (lines 55-59)

Resources Used

1. Build A Web Server With Golang In 20 Mins (2021) - Beginner Friendly!
2. GO And MYSQL - 2024 Project 🚀 💣 🔥 - Connect Go with Mysql / Build a Book …
3. Learn Go: Building a command-line weather app
4. Go & SQL databases: docker compose setup and first queries step by step tutorial
5. How to Read a Text File with Go
6. ChatGPT- GPT-5
7. Cursor - 1.6.45